



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования

«МИРЭА Российский технологический университет»

РТУ МИРЭА

Линтер обучающей версии C++

Выполнил студент 2 курса группы ЭФБО-04-24

Валекжанин Владимир Сергеевич

Москва
2026 г.

Содержание

Содержание.....	2
Введение.....	4
1.1 Цель.....	4
1.2 Предметная область.....	4
1.3 Определения и сокращения.....	4
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	6
1.1 Общее описание.....	6
1.1.1 Описание продукта.....	6
1.1.2 Функции продукта.....	6
1.1.3 Характеристики пользователей.....	6
1.1.4 Ограничения.....	7
1.1.5 Отличия от конкурентов.....	7
1.2 Детальные требования.....	7
1.2.1 Бизнес-требования.....	7
1.2.2 Пользовательские требования.....	8
1.2.3 Функциональные требования.....	8
1.2.4 Нефункциональные требования.....	9
1.2.5 Список задач.....	9
1.3 Диаграммы.....	10
1.3.1 Диаграмма сценариев использования.....	10
1.3.2 Диаграмма последовательности.....	11
1.3.3 Диаграмма классов.....	12
1.4 Применение шаблонов проектирования.....	14
1.4.1 Шаблон «Стратегия» (Strategy).....	14
1.4.1.1 Обоснование применимости.....	14
1.4.1.2 Детальное описание реализации.....	15
1.4.2 Шаблон «Посетитель» в контексте обхода абстрактного синтаксического дерева.....	16
1.4.2.1 Обоснование применимости.....	16
1.4.2.2 Детальное описание реализации.....	16
1.4.3 Шаблон «Фасад».....	17

1.4.3.1 Обоснование применимости.....	17
1.4.3.2 Детальное описание реализации.....	18
1.4.4 Шаблон «Объект контекста».....	19
1.4.4.1 Обоснование применимости.....	19
1.4.4.2 Детальное описание реализации.....	19

Введение

1.1 Цель

Отчет описывает требования к "Линтеру обучающей версии C++". Мы разрабатываем инструмент для автоматической проверки качества кода на C++. Он используется как техническое задание и описание возможностей системы для студентов и преподавателей.

1.2 Предметная область

Линтер нужен для учебного процесса. Он проверяет код на соответствие стандарту C++11 и находит опасные конструкции, которые мешают отладке или вызывают ошибки.

1.3 Определения и сокращения

В тексте используются следующие термины:

- CLI (Command Line Interface) — интерфейс командной строки.
- TDD (Test-Driven Development) — разработка через тестирование.
- Обучающая версия C++ — подмножество стандарта C++11, исключая сложные или потенциально опасные конструкции (например, ручное управление памятью).
- Линтер — программа для анализа кода на ошибки и плохие паттерны.
- AST (Abstract Syntax Tree): Абстрактное синтаксическое дерево — представление синтаксической структуры исходного кода.

- UML (Unified Modeling Language): Унифицированный язык моделирования для визуализации архитектуры системы.
- C++11: Версия стандарта ISO/IEC 14882:2011.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Общее описание

1.1.1 Описание продукта

Данный линтер — это консольное приложение для статического анализа исходного кода. Программа разбирает код без запуска и сверяет его с правилами, чтобы помочь студентам писать чище. Инструмент использует библиотеку Tree—sitter для генерации AST, которое затем обходится движком правил для выявления нарушений.

1.1.2 Функции продукта

Основные задачи линтера:

- Проверка файлов .cpp, .h и .hpp.
- Поиск нарушений правил безопасности и стандартов кода.
- Вывод понятных отчетов в консоль.
- Поддержка модульных правил для легкого расширения.

1.1.3 Характеристики пользователей

У системы три типа пользователей:

- Teacher (Преподаватель) — проверяет работы студентов и контролирует соблюдение учебных норм.

- Student (Студент) — использует линтер для самопроверки и учится писать правильный код.

1.1.4 Ограничения

У проекта есть несколько ограничений:

- Анализ работает только для подмножества C++11.
- Линтер запускается только через командную строку (CLI).
- Программа не исправляет код автоматически — студент должен сделать это сам.
- Сам проект написан на современном стандарте C++20 для надежности.

1.1.5 Отличия от конкурентов

Решение выделяется на фоне промышленных аналогов:

- Инструмент создан специально под учебный курс Программирование корпоративных систем.
- Программа дает пояснения к ошибкам, чтобы студент понимал, в чем проблема.
- Линтер работает быстро и не требует сложной настройки.

1.2 Детальные требования

1.2.1 Бизнес-требования

- Линтер должен следить за соблюдением ограничений курса, чтобы студенты делали меньше типовых ошибок.

- Инструмент должен экономить время преподавателя при проверке стиля и архитектуры работ.

1.2.2 Пользовательские требования

- Программу должно быть легко запустить для одного файла или всей папки через терминал.
- Отчет должен быть ясным: с типом сообщения и точным местом в коде.
- Линтер должен поддерживать коды возврата (exit codes) для работы в скриптах.

1.2.3 Функциональные требования

- Соответствие C++11: Система должна выдавать ОШИБКУ при обнаружении любой возможности, появившейся в C++14 или позже (например, структурированные привязки, бинарные литералы).
- Ручное управление памятью: Система должна выдавать ПРЕДУПРЕЖДЕНИЕ при использовании new или delete.
- Глобальное состояние: Система должна выдавать ПРЕДУПРЕЖДЕНИЕ при объявлении не константных глобальных переменных.
- Приведение типов: Система должна выдавать ПРЕДУПРЕЖДЕНИЕ при использовании приведения типов в стиле C (например, (int)val).
- Соглашения об именовании: Система должна выдавать РЕКОМЕНДАЦИЮ при нарушении соглашений об именовании (например, использование camelCase для переменных).

- Точка с запятой: Система должна выдавать ПРЕДУПРЕЖДЕНИЕ при отсутствии точки с запятой в контекстах, где это требуется соглашениями.
- Парсинг AST: Система должна использовать библиотеку Tree—sitter для точного разбора сложных конструкций C++.

1.2.4 Нефункциональные требования

- Переносимость: Реализация самого линтера должна быть выполнена на стандарте C++20.
- Надежность: Линтер должен разрабатываться с использованием подхода TDD со 100% покрытием всех реализованных правил тестами.
- Удобство использования: Вывод CLI должен включать путь к файлу, номер строки, столбца и описание проблемы.
- Разработка ведется по методике TDD с тестами для каждого правила.

1.2.5 Список задач

Этапы развития проекта:

- Высокий приоритет: Правила по памяти, приведению типов и глобальным переменным через TDD, стабилизация ядра на Tree—sitter.
- Средний приоритет: Цветной вывод в терминале и правила для именования переменных (Naming Conventions).
- Низкий приоритет: Экспорт отчетов в JSON или HTML, «тихий» режим для автоматических проверок

1.3 Диаграммы

1.3.1 Диаграмма сценариев использования

На диаграмме вариантов использования (Рисунок 1.1) показаны основные сценарии взаимодействия пользователей с разрабатываемой системой.

В системе предусмотрены два действующих лица: Студент и Преподаватель. Диаграмма показывает, что им обоим доступны одинаковые функции, которые сводятся к двум основным действиям:

- запуск проверки кода;
- получение отчета об ошибках.

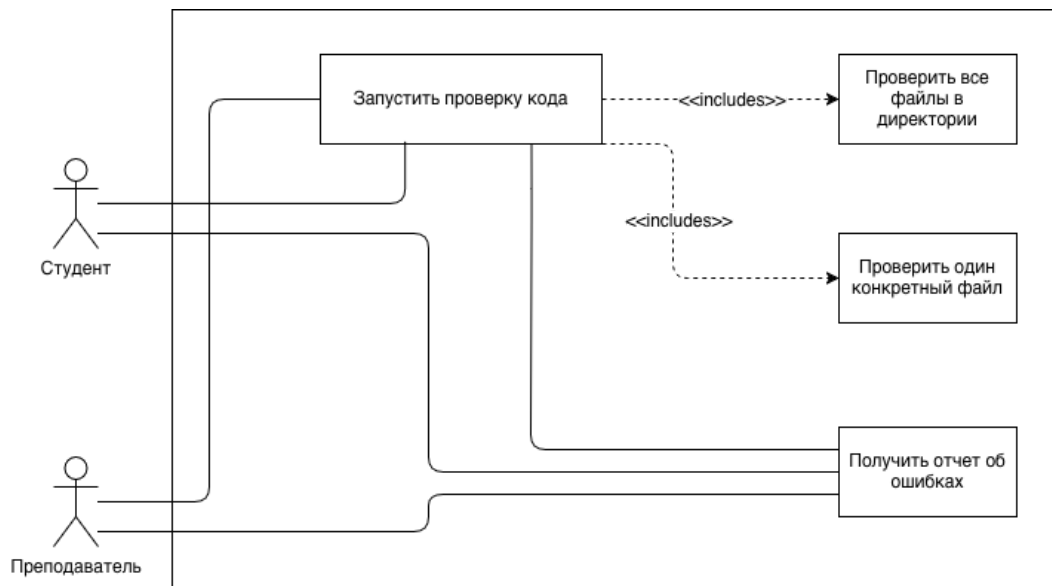


Рисунок 1.1 — Диаграмма сценариев использования

Сценарий запуска проверки дополнительно детализирован с помощью связей включения (includes). При инициализации проверки процесс может пойти по одному из двух путей: встроенные механизмы позволяют проверить

либо сразу все файлы в указанной директории, либо выполнить точечный анализ только одного конкретного файла. После того как любой из этих вариантов отработает, и Студент, и Преподаватель смогут взаимодействовать с системой для получения итогового отчета о найденных в коде недочетах.

1.3.2 Диаграмма последовательности

Процесс начинается с того, что пользователь вводит команду для проверки папки через интерфейс. Это действие запускает главный механизм, который первым делом готовит специальное место для хранения всех найденных проблем. Начальный этап запуска анализа наглядно представлен на рисунке 1.2. Механизм по очереди обходит все файлы с кодом в указанной директории. Для каждого такого файла он просит отдельный инструмент под названием синтаксический анализатор прочитать код и построить его внутреннюю структуру.

Как только внутренняя структура файла готова, механизм передает ее правилам для выполнения проверки. Каждое правило ищет в коде конкретные ошибки или нарушения стиля. Если правило находит проблему, оно записывает эту информацию обратно в общее хранилище, созданное в самом начале. Этот цикл повторяется для каждого узла в дереве кода, чтобы гарантировать точность анализа. После завершения работы с одним файлом программа просто перейдет к следующему, пока не обработает их все.

Когда все файлы проверены, механизм собирает полный список зафиксированных проблем из хранилища. Затем он отправляет этот готовый список обратно в основную консольную программу. Консольная программа использует специальный модуль вывода, чтобы привести результаты к

удобному для чтения виду. В самом конце пользователь видит на экране итоговый отчет со всеми найденными ошибками. На этом работа программы по статическому анализу кода полностью завершается.

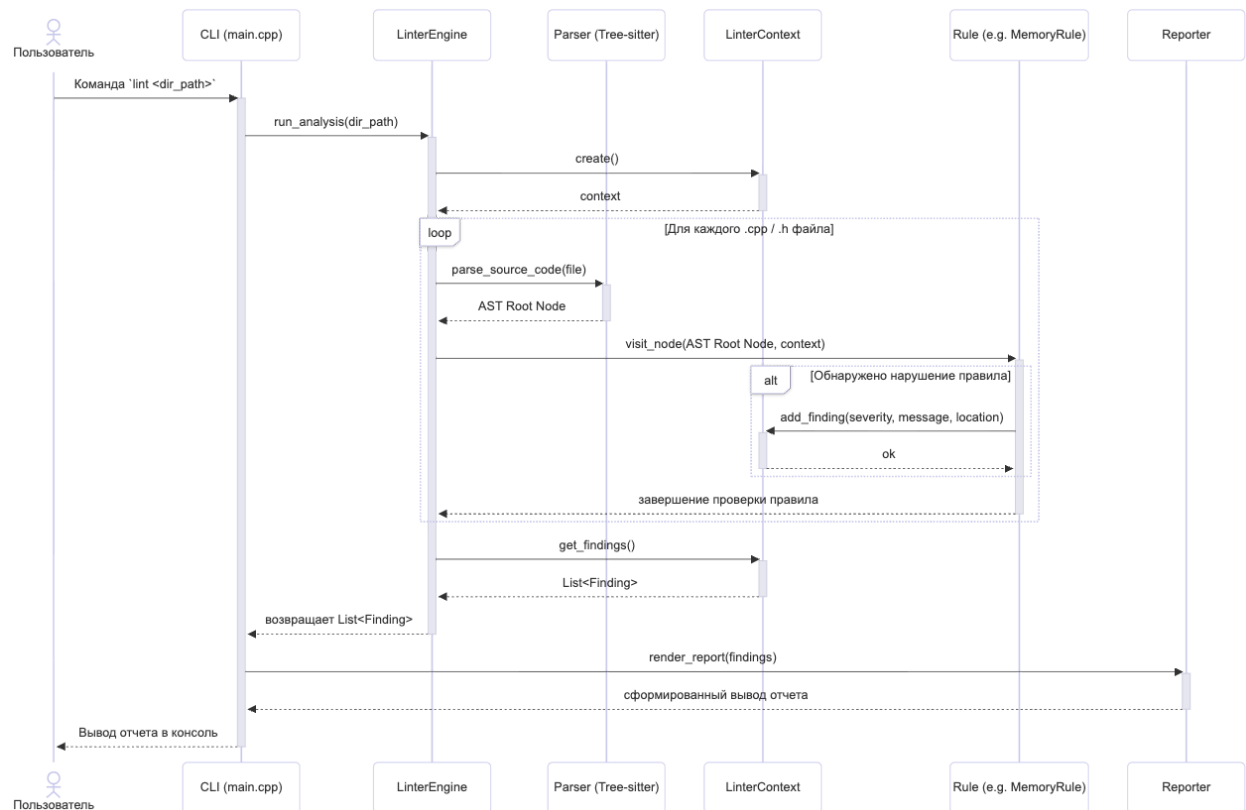


Рисунок 1.2 — Диаграмма последовательности

1.3.3 Диаграмма классов

Основная структура программы строится вокруг центрального модуля управления, который координирует работу всех остальных частей. На рисунке 1.3 видно, как этот главный механизм объединяет в себе синтаксический анализатор и набор различных правил проверки. Он отвечает за запуск процесса анализа и последовательный обход кода. При работе этот модуль создает специальный объект контекста, который сопровождает

проверку каждого файла. Таким образом, ядро системы знает, какие правила нужно применить и где сохранить результаты.

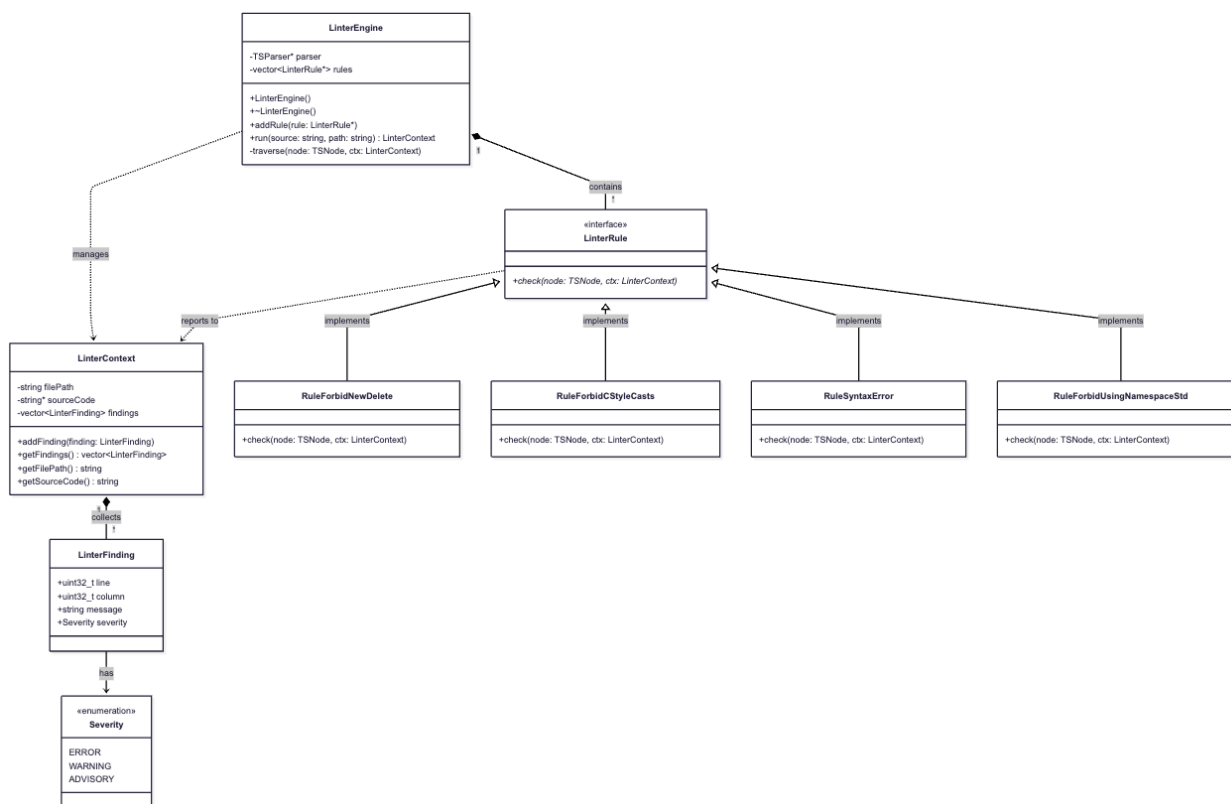


Рисунок 1.3 — Диаграмма классов

Сама логика проверок вынесена в отдельные модули, которые следуют общему стандарту поведения. Существует базовый интерфейс правила, от которого наследуются конкретные реализации, например, для поиска запрещенных функций или ошибок синтаксиса. Каждое такое правило получает на вход фрагмент кода и ссылку на объект контекста. Если правило обнаруживает проблему, оно формирует запись о находке и передает ее в контекст. Такая архитектура позволяет легко добавлять новые требования к коду без изменения существующих частей программы.

Объект контекста служит надежным хранилищем для всех найденных замечаний и метаданных о текущем файле. Каждая зафиксированная находка содержит точные координаты в коде, текст сообщения и уровень важности. Уровень важности определяется через специальное перечисление, которое разделяет ошибки, предупреждения и простые рекомендации. В конце работы вся собранная информация извлекается из контекста для показа пользователю. Это обеспечивает четкое разделение между процессом поиска ошибок и способом их представления.

1.4 Применение шаблонов проектирования

Архитектура линтера опирается на четыре основных шаблона проектирования. Они помогают отделить механизм обхода синтаксического дерева от логики самих проверочных правил. Это делает программу масштабируемой и существенно упрощает дальнейшую поддержку. Ниже приведено подробное описание каждого из внедренных решений.

1.4.1 Шаблон «Стратегия» (Strategy)

Шаблон проектирования «Стратегия» используется для определения семейства алгоритмов проверки кода и их инкапсуляции в виде независимых классов с общим интерфейсом.

1.4.1.1 Обоснование применимости

Анализатор имеет единый механизм перемещения по узлам абстрактного синтаксического дерева, но правила, по которым код проверяется на наличие стилистических и семантических проблем, существенно различаются.

Шаблон «Стратегия» позволяет избежать использования сложных многоуровневых условных конструкций и дает возможность свободно подключать, комбинировать или удалять правила без изменения основного механизма анализа.

1.4.1.2 Детальное описание реализации

В системе спроектирован базовый абстрактный класс Rule, предоставляющий чисто виртуальный метод check(). Конкретные правила унаследованы от данного класса и реализуют свои специфичные механизмы обнаружения (на примере класса RuleForbidCStyleCasts). Подключение стратегий к ядру анализа происходит на этапе настройки анализатора.

Базовый протокол взаимодействия абстрактной стратегии представлен в Листинге 1.1.

Листинг 1.1 — Базовый протокол взаимодействия шаблона «Стратегия»

```
// rule.h
class Rule {
public:
    virtual ~Rule() = default;
    virtual std::string getName() const = 0;
    virtual Severity getSeverity() const = 0;
    virtual void check(TSNode node, LinterContext& ctx) = 0;
};
```

Реализация конкретного алгоритма проверки в рамках стратегии продемонстрирована в Листинге 1.2.

Листинг 1.2 — Конкретная стратегия для проверки приведений типов в стиле языка Си

```
// rule_forbid_c_style_casts.h
class RuleForbidCStyleCasts : public Rule {
public:
    std::string getName() const override { return
«forbid_c_style_casts»; }
```

```
Severity getSeverity() const override { return
Severity::WARNING; }
void check(TSNode node, LinterContext& ctx) override;
};
```

1.4.2 Шаблон «Посетитель» в контексте обхода абстрактного синтаксического дерева

Хотя классический шаблон «Посетитель» подразумевает жесткий механизм двойного вызова, в проекте используется его семантическая адаптация для прохода в глубину узлов абстрактного синтаксического дерева. Данный поведенческий шаблон позволяет добавить новую операцию для узла дерева без изменения его внутренней структуры.

1.4.2.1 Обоснование применимости

Основной цикл работы программы построен на проверке каждого узла дерева. Размещать логику всех проверочных правил (операций) непосредственно в узлах внешней библиотеки грамматического разбора технически невозможно и семантически некорректно. Данный подход позволяет отделить алгоритмы обработки узла от его структурного представления.

1.4.2.2 Детальное описание реализации

В основном классе `LinterEngine` реализован метод `traverse()`. В каждой точке обхода исследуемого узла ядро информирует о нем список активных стратегий. Таким образом, механизм наносит «визит» данным узлам дерева `TSNode`, применяя к ним изменяемый набор проверок.

Алгоритм обхода дерева узлов в глубину запечатлен в Листинге 1.3.

Листинг 1.3 — Механизм шаблона «Посетитель» в процессе обхода абстрактного синтаксического дерева

```
// engine.cpp
void LinterEngine::traverse(TSNode node, LinterContext& ctx) {
    for (const auto& rule : rules) {
        rule->check(node, ctx);
    }

    uint32_t childCount = ts_node_child_count(node);
    for (uint32_t i = 0; i < childCount; ++i) {
        traverse(ts_node_child(node, i), ctx);
    }
}
```

1.4.3 Шаблон «Фасад»

Структурный шаблон проектирования «Фасад» применяется для маскировки сложной структуры внешних библиотек за простым и понятным внешним протоколом взаимодействия.

1.4.3.1 Обоснование применимости

В проекте используется высокоскоростной синтаксический анализатор. Инициализация этой Си-ориентированной библиотеки требует последовательного выделения памяти под анализатор, настройки грамматики языка, преобразования текстовых строк в сущности узлов дерева и, в конечном счете, ручной глубокой очистки памяти. Размещение данной низкоуровневой логики в корневом файле `main.cpp` существенно усложнило бы поддержку проекта.

1.4.3.2 Детальное описание реализации

Сконструирован класс-обертка `LinterEngine`, выступающий упрощенным фасадом для взаимодействия со сторонней подсистемой. Вызывающему коду достаточно передать исходный программный текст в метод `run()`. Ядро самостоятельно управляет жизненным циклом структур языка Си и возвращает полностью сформированный отчет о найденных ошибках в виде объекта контекста проверки.

Протокол взаимодействия спроектированного фасада описан в Листинге 1.4.

Листинг 1.4 — Описание внешнего протокола взаимодействия шаблона «Фасад»

```
// engine.h
class LinterEngine {
public:
    LinterEngine();
    ~LinterEngine() {
        ts_parser_delete(parser);
    }

    void addRule(std::unique_ptr<Rule> rule) {
        rules.push_back(std::move(rule));
    }

    LinterContext run(const std::string& sourceCode, const
std::string& filePath = «inline»);
private:
    void traverse(TSNode node, LinterContext& ctx);

    TSParser* parser;
    std::vector<std::unique_ptr<Rule>> rules;
};
```

1.4.4 Шаблон «Объект контекста»

Шаблон проектирования «Объект контекста» используется для объединения и безопасной передачи состояния выполнения между независимыми логическими слоями, избегая тем самым разрастания списка передаваемых значений и устраняя зависимость от глобальных переменных.

1.4.4.1 Обоснование применимости

Для корректного формирования диагностического сообщения в методе `check()`, стратегиям требуются дополнительные данные: название проверяемого файла, указатель на весь исходный текст для вычисления положения ошибки, а также список для сохранения самих ошибок. Внедрение всех этих полей в описание вызываемого метода привело бы к плохому архитектурному решению «Длинный список передаваемых значений».

1.4.4.2 Детальное описание реализации

Был разработан класс `LinterContext`, собирающий методы для получения сопутствующих данных и предоставляющий набор средств для фиксации найденных недочетов. Экземпляр контекста инициализируется единожды для каждого анализируемого файла и каскадно пробрасывается всем задействованным правилам через шаблон обхода дерева. Сформированный массив ошибок извлекается фасадом на финальной стадии работы.

Реализация объекта контекста отображена в Листинге 1.5.

Листинг 1.5 — Структура класса в рамках шаблона «Объект контекста»

```
// rule.h
class LinterContext {
```

```

public:
    explicit LinterContext(std::string filePath, const
std::string* sourceCodePtr) :
                                filePath(std::move(filePath)),
sourceCodePtr(sourceCodePtr) {}

    void addFinding(Finding f) {
        findings.push_back(std::move(f));
    }

    const std::vector<Finding>& getFindings() const { return
findings; }

private:
    std::string filePath;
    std::vector<Finding> findings;
    const std::string* sourceCodePtr;
};

```